

scRapEU: An R function to scrape data on EU laws

Michele Scotto di Vettimo*

2022-01-12

Description

scRapEU is an R function that can be used to automatically download data about procedure files available on the [European Union's Legislative Observatory \(OEIL\)](#). It downloads most of the information available on the procedure page (see below), complements it with data from the [EUR-Lex](#) website, and stores it in a dataframe that can subsequently be saved by the user in the preferred format.

Details

Access

The function can be accessed directly from the R workspace as follows:

```
source("https://michelescottodivettimo.eu/scrapeu/scRapEU.R") # Import function
```

Alternatively, the code can be downloaded from [this link](#),

Arguments

The function has few arguments that help the user to refine the search criteria.

exact_procedures Use this argument to specify the procedure number of the file(s) to download. The input should be a character vector as the following: `c("1999/0003(COD)", "2018/0330A(COD)")`. If not NULL, the argument overrides all the other search criteria.

years Defines the years to be downloaded.

procedures Defines the procedure types to be searched for. Possible values are the abbreviated names of the procedure types as per the [OEIL](#) naming conventions. By default, it scrapes all procedure types.

verbose If TRUE, print progress messages as the scraper goes.

Output

The function returns a dataframe for every year searched, where each row represents a procedure file. If more years are searched in the same call, the dataframes are stored in a list.

Variables stored

Procedure number Unique identifier of the dossier.

procedure_title Main title of the procedure on OEIL.

*Department of Political Economy, King's College London.

proposal__number Number of the initiating document.

celex__proposal CELEX number of the initiating document.

date__initiation Date of the initiating document.

proposal__title Title of the initiating document.

procedure__type Type of procedure (e.g., codecision, consultation, etc.)

procedure__subtype Subtype of the procedure (e.g., legislation, codification, etc.)

procedure__status Status of the procedure in the decision-making process (e.g., completed, pending, withdrawn, rejected)

revised__proposal Dummy indicating if a revised proposal has been issued or not.

date__end End of conclusion of the procedure. NA if still pending.

celex__final CELEX number of the final document. NA if not completed.

act__name Short name of the final document (e.g., Regulation 2019/1896). NA if not completed.

oj__reference Reference number on the Official Journal (e.g., OJ L 248 13.07.2021, p. 0011). NA if not completed.

leg__instrument Legal instrument used in the procedure.

subject__oeil Subject codes available on OEIL basic information section.

eurovoc__desc EUROVOC descriptors of the initiating document on EUR-Lex.

directory__code Directory code number. See [this link](#).

directory__code__names Names associated to the directory code of the initiating document on EUR-Lex.

subject__matter Subject matter elements of the initiating document on EUR-Lex.

cap__topics Policy area classification following the EU Policy Agenda Project (EU-PAP) categories. The variable matches the value of **subject__oeil** variable with policy categories from the EUPAP coding scheme. The scheme is basically an adaptation of the Comparative Agendas Project classification to the European Union. See *Supplementary resources* section below.

ep__cmtee Name of the European Parliament committee(s) responsible for the procedure.

ep__cmtee__opinion Name of the European Parliament committee(s) asked for opinion on the procedure.

n_ep__cmtee__opinion Number of European Parliament committee(s) asked for opinion on the procedure

rapporteur Name of the rapporteur assigned to the procedure.

rapporteur__appnt Appointment date of the rapporteur assigned to the procedure

rapporteur__party Political party of the rapporteur.

rapporteur__url Link to the rapporteur's webpage on the EP website.

shadow__rapporteur Names of shadow rapporteurs for the procedure (in development)

cmtee__dossier Number assigned to the EP committee dossier of the procedure.

plenary__texts Serial number of texts related to the procedure debated in the European Parliament.

texts__adopted Text adopted by the European Parliament related to the procedure.

council__config__oeil Council configurations debating the procedure (from OEIL).

council__session__id__oeil Serial number identifying the Council sessions debating the procedure (from OEIL).

council_session_date_oeil Dates of the Council sessions debating the procedure (from OEIL).

council_config_eurlex Council configurations debating the procedure (from EUR-Lex).

council_session_date_eurlex Dates of the Council sessions debating the procedure (from EUR-Lex).

council_session_id_eurlex Serial number identifying the Council sessions debating the procedure (from EUR-Lex).

council_session_scope_eurlex Scope of the Council sessions debating the procedure (from EUR-Lex).

council_agenda_item_eurlex Type of agenda item on Council agenda (from EUR-Lex).

trilogue Dummy recording reference to inter-institutional negotiations in list of key events.

legal_basis Articles of the EU treaties and institutions' rules of procedures referred to as legal basis of the procedure being scraped.

commission_dg Commission Directorate-General responsible for the proposal.

commissioner Commissioner responsible for the proposal.

natparl_contribution National parliaments sending contributions regarding the proposal.

natparl_doctype Type of national parliament contribution.

natparl_docref Reference number of documents associated to national parliament contributions.

natparl_docdate Dates associated to national parliament contributions.

summary_proposal_id Identified of the summary of the initiating document.

summary_final_id Identifier of the summary of the final act. NA if not completed.

key_events List of key events associated to the procedure.

key_dates Dates of the key events listed.

date_scraped Date of the scraping.

Minimal examples

By adapting the function's argument, the user can refine the search criteria in different ways.

The following code shows how to extract procedures based on their procedure numbers:

```
scrape_this <- c("2018/0330A(COD)", "2002/0024(COD)") # Vector of procedures to be searched

cods <- scRapEU(exact_procedures = scrape_this) # Call scraping function

view(cods) # View dataframe
```

The `cods` object will then be a single dataframe with two rows representing the selected procedures.

The user can also specify the year(s) and the types of procedures to be scraped. The following lines of code show two examples where one or more years are specified. It is important to note that the number of years scraped affects the way the dataframes are stored and returned after the execution and, thus, the way they can be accessed afterwards.

```
# To scrape all codecision files in 1999
cod99 <- scRapEU(years = 1999, procedures = "COD")

# To scrape all codecision files in 1999 and 2000
cod99_00 <- scRapEU(years = c(1999:2000), procedures = "COD")
```

```
## cod99 is already a dataframe object. Instead, cod99_00 is a list with two dataframes.

# To access the 1999 data stored in cod99_00, the user can do the following:
cod99b <- cod99_00$df1999
```

As the function returns the scraped data only at the end of the execution, when scraping multiple years it might be preferable to use the function in a loop to scrape (and save) one year at the time. In this way completed years can be accessed before the end of the last download.

```
# Loop over the desired years and save each dataframe in a single file

for (year in c(1994:2025)){ # Wow! 30 years of EU laws. How exciting!
  cod_df <- scRapEU(years = year, procedures = "COD") # Get procedures
  saveRDS(cod_df, file = paste0("cod_files", as.character(year), ".RDS")) # Save RDS file
}

# You can now merge all .RDS files in a single dataframe using rbind().
# See R documentation for exporting a dataframe to Excel or Stata format.

rds_files <- list.files(pattern = 'cod_files*.RDS') # Get all files in the directory
all_eu_laws <- NULL # Empty object to populate
for (r in 1:length(rds_files)){ # Loop over files to append to object
  all_eu_laws <- rbind(all_eu_laws, readRDS(rds_files[r]))
}
dim(all_eu_laws) # That's a fairly big dataset!
```

Debugging and re-use

Common errors and debugging

The function is not error free. In most cases, though, it should run smoothly despite retrieving NA values instead of true information. Yet, there might be errors that prevent the execution of the code. Common errors that might occur have to do with accessing the HTML page of the procedure. If this happens, just re-run the function, as these errors have to do with the scraped websites or the internet connection. Also, I advise against the use of VPNs while running the web scraper.

At the moment the script can deal with some of these errors by re-attempting the connection to the webpage 5 times with a short time interval between each attempt. If after the last attempt the page cannot be accessed, the procedure will still be stored in the dataframe, but all the information will have NA values.

If you have reasons to believe that the error cannot be traced back to the webpage, then you can either flag it up to me or edit the R code by yourself. In such event, re-running the function with the argument `verbose = TRUE` might help in spotting where the error is occurring. In any case, when the function crashes it should by default print information about the type of error and the line(s) of code that triggered it. If the function does not incur in the same error again and, instead, runs smoothly, it might be that the problem was in fact related to the internet connection.

Supplementary resources

There are ancillary scripts and datasets that are used to expand on the variables automatically downloaded by `scRapEU`. Those are called by the function itself internally, so the user does not need to interact with them directly. However, they might be of some interest and can be used outside of the scope of this R function.

More specifically, at the moment these additional resources deal with the matching of OEIL subject descriptors with more established policy area classifications.

Finally, there is also a short report available at [this link](#) giving an overview of the data extracted using the script and covering the procedures initiated between 1999 and 2025. The report also discusses issues related

to missing data in the downloaded datasets.

Matching OEIL subjects with EU-PAP policy categories To classify procedures according to the policy area classification from the EU-PAP project, I start from the `subject_oel` variable and manually matched each value to the corresponding EU-PAP subtopic, which are in turn nested within 21 distinct topics (see [codebook](#)). A spreadsheet with the matching matrix is available at [this link](#).